A production-ready, API-first order management system built with Laravel — open sourced from the internal stack powering client projects at (SFG).
The same system i build and deploy for e-commerce and retail businesses, free version for anyone to use, fork, and extend.



Features • Architecture • Installation • API Documentation • Testing • Roadmap

---

📌 Table of Contents

- Overview
- Features
- Technology Stack
- Architecture
- Requirements
- Installation
- Environment Configuration
- Database Setup
- Running the Application
- Authentication
- API Documentation
- Order Lifecycle
- Inventory Management
- Background Jobs
- Performance and Scalability
- Testing
- Project Structure
- Security Considerations
- Roadmap
- Contributing
- License

🎯 Overview

The **Order Management System (OMS)** is a backend application built with Laravel that provides RESTful APIs for managing products, inventory, customers, vendors, and orders.

It is designed around a modular architecture that separates business logic, data access, HTTP handling, and asynchronous processing.

💡 What Problem Does It Solve?

Businesses need a reliable way to manage their product catalogs, process customer orders, track stock levels, and coordinate fulfillment.

This system brings those operations together through a centralized API.

👥 User Roles

Role	Responsibilities
🌓 Admin	Manage products, oversee orders, monitor inventory, and administer the platform.
🏪 Vendor	Manage authorized products, update fulfillment status, and monitor stock.
🛍️ Customer	Browse products, place orders, track order status, and cancel eligible orders.

Access must be enforced through server-side authorization policies, not merely through frontend restrictions.

✨ Features

📦 Product & Inventory Management

- ☑ Create, retrieve, update, and delete products.
- ☑ Support product variants.
- ☑ Track stock quantities.
- ☑ Identify products with low inventory.
- ☑ Import products in bulk using CSV files.
- ☑ Search products by name and description.
- ☑ Restrict product management by user role.

🛒 Order Processing

- ☑ Create orders containing multiple products.
- ☑ Track orders through a defined status workflow.
- ☑ Deduct inventory when an order is confirmed.
- ☑ Restore inventory when an eligible order is cancelled.
- ☑ Generate PDF invoices.
- ☑ Send order-related email notifications.
- ☑ Retrieve orders according to user permissions.

🔐 Authentication & Authorization

- ☑ JWT-based authentication.
- ☑ Token refresh support.
- ☑ Protected API endpoints.
- ☑ Role-based access control.
- ☑ Request validation.
- ☑ Configurable API rate limiting.

Background Processing

- [x] Queue-based email notifications.
- [x] Asynchronous invoice generation.
- [x] Scheduled low-stock checks.
- [x] Event-driven business workflows.

API & Engineering

- [x] Versioned API structure.
- [x] Service-layer business logic.
- [x] Repository pattern for data access.
- [x] Dedicated actions for complex operations.
- [x] Database query optimization.
- [x] Redis-backed caching and queues.
- [x] Unit and feature testing.
- [x] PSR-12 coding standards.

> **Implementation note:** Feature descriptions reflect the intended system design. Confirm that each feature has been implemented and tested in the repository before treating it as production-ready.

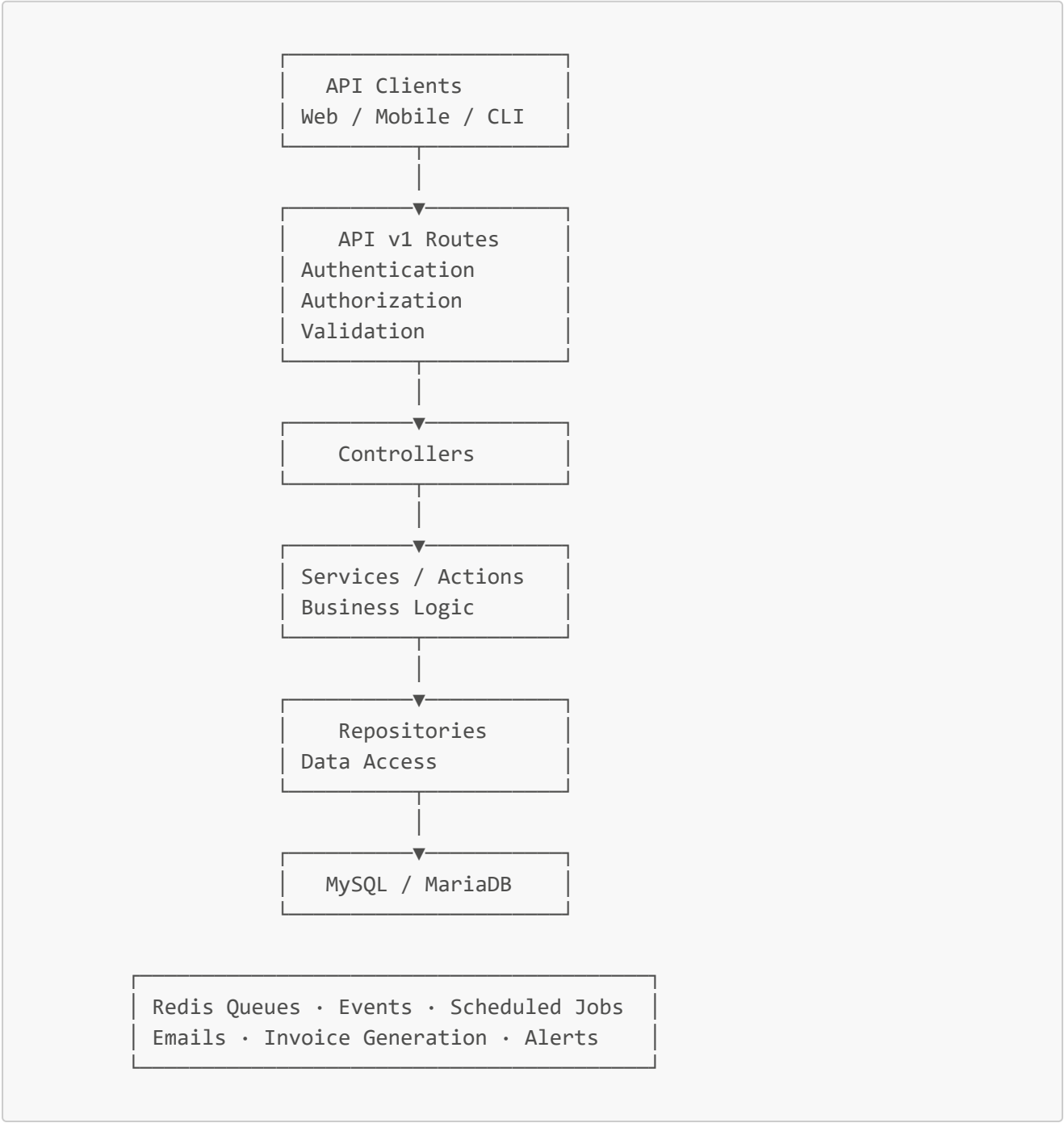
Technology Stack

Technology	Purpose
PHP 8.2+	Backend programming language
Laravel 10+	Application framework
MySQL / MariaDB	Relational database
Redis	Caching and queue processing
JWT	API authentication
DomPDF	PDF invoice generation
Laravel Excel	Spreadsheet and CSV processing, if used by the implementation
PHPUnit / Pest	Automated testing
Composer	PHP dependency management
Node.js & npm	Optional frontend tooling

Compatibility note: Laravel 10 supports PHP 8.1 and above, but this project targets PHP 8.2+. Laravel 10 is no longer receiving security fixes under its original support policy. For a new production deployment in 2026, consider upgrading to a currently supported Laravel release after verifying package compatibility.

Architecture

The application follows a layered architecture that keeps HTTP controllers thin and isolates business operations.



Architectural Components

Controllers

Handle HTTP requests, invoke application logic, and return structured responses.

Services

Coordinate business rules and reusable operations.

Actions

Encapsulate specific workflows, such as processing orders, cancelling orders, and importing products.

Repositories

Centralize data access and provide a consistent interface for querying and persisting data.

Events and Listeners

Decouple business operations from secondary effects such as notifications.

Jobs

Move suitable long-running or noncritical tasks out of the request-response cycle.

Why This Architecture?

- **Maintainability:** Business logic is separated from HTTP concerns.
- **Testability:** Services and actions can be tested independently.
- **Reusability:** Multiple controllers can use the same application services.
- **Scalability:** Background work can be processed by separate workers.
- **Extensibility:** New modules can be added without putting all logic in controllers.

Use these patterns where they add value. An abstraction should simplify the application, not create unnecessary layers.

📋 Requirements

Before installing the application, ensure your development environment includes:

- PHP 8.2 or later, compatible with the selected Laravel release.
- Composer 2.x.
- MySQL 5.7+ or a compatible MariaDB version.
- Redis, if Redis is configured for caching or queues.
- Required PHP extensions for Laravel and installed dependencies.
- Git.
- Node.js and npm, if frontend assets are required.

> **Important:** MySQL 5.7 is end-of-life. For a new deployment, use a currently supported MySQL or MariaDB release and verify compatibility with your application dependencies.

Check your environment:

```
php -v
composer --version
php -m
mysql --version
redis-cli ping
```

The last two commands require the relevant clients to be installed. A successful Redis connection should return `PONG`.

🚀 Installation

1. Clone the Repository

Replace the placeholder with your actual repository URL.

```
git clone <your-repository-url>
cd oms-api
```

2. Install Dependencies

```
composer install
```

If Composer reports that an extension such as `ext-zip` is missing, install and enable that extension before continuing.

Avoid using `--ignore-platform-reqs` as a permanent workaround. It can allow installation to proceed while leaving the application unable to run correctly.

3. Configure Environment Variables

Create your local environment file:

```
cp .env.example .env
```

On Windows Command Prompt, you can use:

```
copy .env.example .env
```

Update the database, Redis, application URL, and mail settings in `.env`.

4. Generate the Application Key

```
php artisan key:generate
```

5. Configure JWT Authentication

If the application uses `tymon/jwt-auth`, generate the JWT signing secret:

```
php artisan jwt:secret
```

Run this command only after installing the package and configuring the relevant authentication settings.

6. Run Database Migrations

```
php artisan migrate
```

7. Seed the Database

```
php artisan db:seed
```

If the included seeders match the configuration described for this project, the development database will contain:

- 3 roles: Admin, Vendor, and Customer.
- 1 administrator.
- 3 vendors.
- 5 customers.
- 15 sample products with inventory records.

Use only disposable development databases for demo seeders.

8. Start the Queue Worker

```
php artisan queue:work
```

Keep the worker running in a separate terminal.

9. Start the Development Server

```
php artisan serve
```

The default base URL is:

```
http://localhost:8000/api/v1
```

Installation complete. You can now test the available API endpoints using Postman, Insomnia, or `curl`.

Environment Configuration

Configure the following variables in your `.env` file.

```
APP_NAME="Order Management System"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=oms_api
DB_USERNAME=root
DB_PASSWORD=

REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

QUEUE_CONNECTION=redis
CACHE_DRIVER=redis

JWT_TTL=60
JWT_REFRESH_TTL=20160

LOW_STOCK_THRESHOLD=10
API_RATE_LIMIT=60

PDF_ENABLED=true
```

Configuration Notes

- Set `APP_DEBUG=false` in production.
- Use strong, unique database credentials outside development.
- Verify the correct environment variable names against the Laravel version and installed packages.
- Laravel 10 commonly uses `CACHE_DRIVER`; newer Laravel versions may use `CACHE_STORE`.
- The JWT lifetime variables must match the JWT package configuration.
- Custom variables such as `LOW_STOCK_THRESHOLD` and `PDF_ENABLED` must be explicitly read by the application.
- Never commit your real `.env` file or signing secrets.

Database Setup

Create the database before running migrations if it does not already exist.

```
CREATE DATABASE oms_api
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Then run:

```
php artisan migrate --seed
```

Suggested Core Entities

Entity	Responsibility
`users`	User accounts and authentication
`roles`	Available application roles
`products`	Product catalog
`product_variants`	Product variations, where implemented
`inventory`	Stock quantities and inventory records
`orders`	Order headers and status
`order_items`	Products, quantities, and prices within orders

The exact schema depends on the migrations in your repository.

Important Data Integrity Rules

For a reliable order management system:

- Store prices using an appropriate fixed-precision decimal representation or integer minor currency units.
- Preserve the price charged for each product in the order item.
- Enforce unique SKU constraints where required.
- Use foreign keys for important relationships.
- Prevent negative inventory through database-safe updates and transaction handling.
- Keep an inventory movement history if stock auditing is required.

🔐 Authentication

The API uses JWT-based authentication when the JWT package and authentication guards are configured accordingly.

Register a Customer

```
POST /api/v1/auth/register
Content-Type: application/json
Accept: application/json
```

Request:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "use-a-strong-password",
  "password_confirmation": "use-a-strong-password"
}
```

The registration endpoint should assign the customer role on the server. Do not allow public clients to choose privileged roles by submitting a `role_id`.

Login

```
POST /api/v1/auth/login
Content-Type: application/json
Accept: application/json
```

Request:

```
{
  "email": "john@example.com",
  "password": "use-a-strong-password"
}
```

Illustrative response:

```
{
  "message": "Login successful",
  "data": {
    "user": {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com"
    },
    "token": "<access-token>",
    "token_type": "bearer",
    "expires_in": 3600
```

```
    }
  }
```

The response structure depends on the actual controller implementation.

Authenticate Subsequent Requests

Include the access token in the request headers:

```
Authorization: Bearer <access-token>
Accept: application/json
```

Refresh a Token

```
POST /api/v1/auth/refresh
Authorization: Bearer <access-token>
Accept: application/json
```

Logout

```
POST /api/v1/auth/logout
Authorization: Bearer <access-token>
Accept: application/json
```

The application must implement token invalidation and refresh behavior according to the selected JWT package.

API Documentation

All endpoints use the `/api/v1` prefix.

The following table describes the intended API surface. Actual routes, request validation, response schemas, and authorization rules should be confirmed against `routes/api.php` and the relevant controllers.

Authentication

Method	Endpoint	Description
`POST`	`/auth/register`	Register a customer
`POST`	`/auth/login`	Authenticate a user

Method	Endpoint	Description
POST	/auth/logout	Invalidate the current token
POST	/auth/refresh	Refresh the access token
GET	/auth/me	Retrieve the authenticated user

Products

Method	Endpoint	Access
GET	/products	Public or authenticated, according to configuration
GET	/products/{id}	Public or authenticated, according to configuration
POST	/products	Admin / authorized vendor
PUT	/products/{id}	Admin / authorized vendor
DELETE	/products/{id}	Admin / authorized vendor
POST	/products/import	Admin / authorized vendor

Orders

Method	Endpoint	Access
GET	/orders	Authenticated; results scoped to permissions
GET	/orders/{id}	Authorized order participants
POST	/orders	Customer
PUT	/orders/{id}/status	Admin / authorized vendor
POST	/orders/{id}/cancel	Authorized order participant

Inventory

Method	Endpoint	Access
GET	/inventory/{id}	Admin / authorized vendor
PUT	/inventory/{id}	Admin / authorized vendor
GET	/inventory/low-stock/list	Admin / authorized vendor

Example: Retrieve Products

```
curl -X GET "http://localhost:8000/api/v1/products" \
  -H "Accept: application/json"
```

Example: Create an Order

```
POST /api/v1/orders
Authorization: Bearer <access-token>
Content-Type: application/json
Accept: application/json
```

Illustrative request:

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 2
    },
    {
      "product_id": 2,
      "quantity": 1
    }
  ]
}
```

The server should calculate prices, validate stock, and determine the final order total. Never trust client-supplied totals or prices.

Recommended API Improvements

For a production-ready API, document:

- Pagination and sorting parameters.
- Search and filtering parameters.
- Request and response schemas.
- Validation error formats.
- Authentication requirements.
- Role and ownership restrictions.
- HTTP status codes.
- Rate-limit responses.
- Idempotency behavior for order creation.

Consider generating an OpenAPI specification and publishing interactive API documentation.

Order Lifecycle

The intended order workflow is:

```
PENDING
   │
   ▼
PROCESSING
   │
   ▼
SHIPPED
   │
   ▼
DELIVERED
```

Cancellation is a separate transition available only when permitted by the business rules.

```
PENDING ──────────► CANCELLED
PROCESSING ───────► CANCELLED
```

Whether an order can be cancelled after shipping should be an explicit business rule.

Inventory Consistency

Order processing should use database transactions to keep order records and stock updates consistent.

A reliable workflow should:

1. Validate the order items.
2. Retrieve the authoritative product prices.
3. Lock or safely update relevant inventory records.
4. Verify that sufficient stock is available.
5. Deduct inventory according to the defined confirmation policy.
6. Create the order and its items.
7. Commit the transaction.
8. Dispatch notifications after successful processing.

Use row-level locking or conditional atomic updates to prevent concurrent requests from overselling stock.

Cancellation and Rollback

Cancellation must restore inventory only if that inventory was previously deducted.

Use an explicit inventory reservation or deduction state, and make cancellation idempotent. Otherwise, repeated cancellation requests can restore the same stock more than once.

📦 CSV Product Import

The bulk-import endpoint is:

```
POST /api/v1/products/import
Authorization: Bearer <access-token>
Content-Type: multipart/form-data
```

Expected CSV Format

```
name,description,price,sku,quantity
Product 1,Description 1,99.99,SKU001,100
Product 2,Description 2,149.99,SKU002,50
```

Import Validation

A robust importer should:

- Validate file type and size.
- Validate required columns.
- Reject malformed prices and quantities.
- Detect duplicate SKUs.
- Report row-level validation errors.
- Process large files in chunks.
- Restrict imports to authorized users.
- Use transactions or controlled batch commits.
- Avoid partially importing data without reporting failures.

For large catalogs, queue the import rather than processing the entire file in a single HTTP request.

Background Jobs

Use Laravel queues for operations that do not need to block the main HTTP response.

Job	Purpose
`SendOrderEmailJob`	Send order-related email notifications
`GenerateInvoicePdfJob`	Generate PDF invoices
`CheckLowStockJob`	Detect products below configured stock thresholds

Start a Worker

```
php artisan queue:work
```

Run Scheduled Tasks

Register the low-stock check in Laravel's scheduler according to your application version.

On a server, configure the scheduler to run every minute:

```
* * * * * cd /path/to/oms-api && php artisan schedule:run >> /dev/null 2>&1
```

The actual schedule should be defined in the application, for example, hourly for low-stock checks.

Production Considerations

- Configure failed-job storage.
- Implement retries and sensible backoff.
- Make jobs safe to retry.
- Monitor queue latency.
- Run workers under a process manager.
- Dispatch notifications after successful database commits.

🚀 Performance and Scalability

1. Database Optimization

- Eager-load relationships to prevent N+1 queries.
- Add indexes based on real query patterns.
- Paginate product and order listings.
- Use appropriate database constraints.
- Profile slow queries before optimizing them.
- Avoid loading entire catalogs into memory.

2. Redis Caching

Potential caching targets include:

Data	Suggested TTL
Product details	1 hour
Product listings	5–15 minutes
Role and permission data	15–60 minutes
Inventory summaries	Short TTL or event-driven invalidation
Low-stock reports	5–15 minutes

Inventory and order data require special care because stale values can mislead users.

Example:

```php
use Illuminate\Support\Facades\Cache;

$cacheKey = "products:all:{$page}:{$perPage}";

return Cache::remember(
    $cacheKey,
    now()->addMinutes(15),
    fn () => $this->repository->paginate($perPage)
);
```

The repository method and pagination parameters should match your actual implementation.

3. Cache Invalidation

Invalidate relevant cache entries when:

- A product is created, updated, or deleted.
- Inventory quantities change.
- An order is created or cancelled.
- A user's permissions change.

Prefer targeted invalidation, versioned cache keys, or cache tags where supported by the configured driver.

4. Rate Limiting

Suggested starting limits:

Client	Requests per minute
Anonymous	10
Authenticated	60
Administrator	120

These are illustrative limits, not universal defaults. Tune them according to traffic, endpoint cost, and business requirements.

Apply stricter limits to authentication, password recovery, and expensive operations such as CSV imports.

5. Horizontal Scaling

As traffic grows, consider:

- Multiple stateless API instances.
- A shared Redis service.
- Dedicated queue workers.
- Database read replicas for appropriate read workloads.
- Object storage for generated invoices.

- Centralized logs and metrics.
- Load balancing and health checks.

6. Database Sharding

Sharding can help at very large scale, but it introduces operational complexity.

Before sharding, evaluate whether indexing, query optimization, read replicas, and better workload separation can meet the required capacity.

If sharding becomes necessary:

- Choose a shard key based on access patterns.
- Keep related order and order-item data colocated where practical.
- Plan for cross-shard reporting.
- Design migration and rebalancing procedures.
- Handle transactions that span multiple shards explicitly.

Do not treat sharding as a default requirement for an ordinary order management system.

🧪 Testing

Run the test suite:

```
php artisan test
```

Run feature tests:

```
php artisan test --testsuite=Feature
```

Run unit tests:

```
php artisan test --testsuite=Unit
```

Generate coverage where the required coverage driver is installed and configured:

```
php artisan test --coverage
```

Recommended Test Coverage

Authentication

- Valid and invalid login attempts.
- Token refresh and logout.
- Registration validation.
- Privilege escalation prevention.

Products

- CRUD operations.
- Vendor ownership restrictions.
- Search and pagination.
- CSV validation and duplicate SKU handling.

Orders

- Multi-item order creation.
- Insufficient inventory.
- Concurrent order attempts.
- Valid and invalid status transitions.
- Cancellation and inventory restoration.
- Duplicate cancellation requests.

Inventory

- Stock updates.
- Low-stock thresholds.
- Concurrent stock changes.
- Authorization checks.

Background Jobs

- Email dispatch.
- Invoice generation.
- Retry behavior.
- Failed-job handling.

> A successful test command does not automatically prove production readiness. Add integration tests for the database, Redis, queues, and any external services your deployment depends on.

Project Structure

The following is the intended structure. Actual files may differ depending on the current repository.

```
oms-api/
├── app/
│   ├── Actions/
│   │   ├── ProcessOrderAction.php
│   │   ├── CancelOrderAction.php
│   │   └── ImportProductsAction.php
│   │
```

```
│   ├── Events/
│   │   ├── OrderCreated.php
│   │   ├── OrderStatusChanged.php
│   │   └── LowStockDetected.php
│   │
│   ├── Http/
│   │   ├── Controllers/
│   │   │   └── Api/
│   │   │       └── V1/
│   │   │           ├── AuthController.php
│   │   │           ├── ProductController.php
│   │   │           ├── OrderController.php
│   │   │           └── InventoryController.php
│   │   ├── Middleware/
│   │   │   └── CheckRole.php
│   │   ├── Requests/
│   │   └── Resources/
│   │
│   ├── Jobs/
│   │   ├── SendOrderEmailJob.php
│   │   ├── GenerateInvoicePdfJob.php
│   │   └── CheckLowStockJob.php
│   │
│   ├── Listeners/
│   ├── Models/
│   ├── Repositories/
│   │   └── Contracts/
│   └── Services/
│
├── bootstrap/
├── config/
├── database/
│   ├── factories/
│   ├── migrations/
│   └── seeders/
│
├── routes/
│   └── api.php
│
├── tests/
│   ├── Feature/
│   └── Unit/
│
├── .env.example
├── composer.json
├── phpunit.xml
└── README.md
```

Security Considerations

Security must be enforced on the backend.

- **Authentication:** Protect private routes with the configured authentication guard.
- **Authorization:** Verify both role permissions and resource ownership.
- **Mass assignment:** Use validated data and carefully define fillable or guarded model attributes.
- **Input validation:** Validate every request, including CSV imports.
- **Secrets:** Never commit JWT secrets, database passwords, or production credentials.
- **Rate limiting:** Protect login, registration, and resource-intensive endpoints.
- **Inventory integrity:** Use transactions and concurrency-safe stock updates.
- **Error handling:** Avoid exposing stack traces or sensitive database details in production.
- **Logging:** Never log passwords, access tokens, or other secrets.
- **Dependencies:** Audit packages and upgrade unsupported framework versions.
- **HTTPS:** Use TLS for all production API traffic.

Production Deployment Checklist

- ☐ `APP_DEBUG=false`
- ☐ Strong, unique application and JWT secrets.
- ☐ Database credentials secured.
- ☐ Redis protected from unauthorized access.
- ☐ Migrations reviewed and tested.
- ☐ Queue workers supervised.
- ☐ Scheduler configured.
- ☐ Logs and failed jobs monitored.
- ☐ Backups and recovery procedures tested.
- ☐ Authentication and authorization tests passing.
- ☐ Supported PHP and Laravel versions selected.
- ☐ API documentation reviewed against actual routes.

🗺 Roadmap

Potential future improvements:

- ☐ Interactive OpenAPI / Swagger documentation.
- ☐ Automated CI/CD with GitHub Actions.
- ☐ Docker-based development environment.
- ☐ Inventory movement audit trail.
- ☐ Idempotency keys for order creation.
- ☐ Warehouse-level inventory management.
- ☐ Payment gateway integration.
- ☐ Order tracking and fulfillment integrations.
- ☐ Advanced reporting and analytics.
- ☐ Observability with structured logs and metrics.
- ☐ Load testing and performance benchmarks.
- ☐ Multi-tenant vendor isolation.

Items on this roadmap are proposed enhancements, not claims about existing functionality.

🤝 Contributing

Contributions are welcome.

1. Fork the repository.
2. Create a feature branch.
3. Make your changes.
4. Add or update tests.
5. Run the test suite.
6. Commit your changes with a clear message.
7. Open a pull request describing the changes.

Keep pull requests focused and include tests for changes to business logic, authorization, and inventory processing.

👨‍💻 Author

Your Name

- GitHub: @yourusername
- Email: your.email@example.com

Replace these placeholders with your actual details.

📄 License

This project is licensed under the MIT License, if the repository includes the corresponding MIT license file.

See the `LICENSE` file for details.

🙏 Acknowledgments

Built with the help of the Laravel and PHP ecosystems.

- Laravel — Application framework.
- tymon/jwt-auth — JWT authentication, if used by the implementation.
- barryvdh/laravel-dompdf — PDF generation, if installed.
- Maatwebsite Laravel Excel — Spreadsheet processing, if installed.

Built for reliable order processing, consistent inventory, and maintainable backend systems.

Designed with clean architecture, security, and scalability in mind.